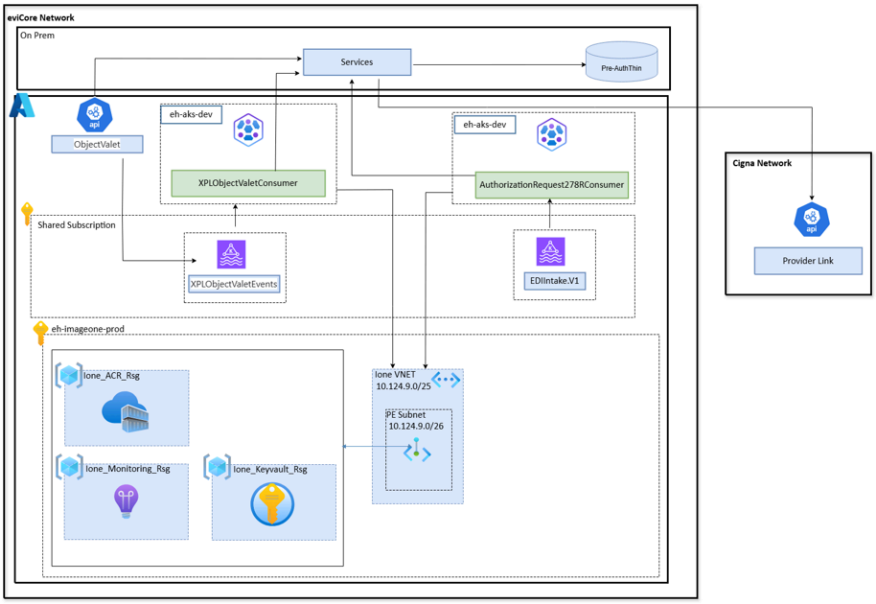
eP Convergence - AuthorizationRequest278R Observer and XPLObjectValetEvents Observer + Technical Document

Overview :

This page provides details around the observer that will listen to events raised for AuthorizationRequest278R and XPLObjectValetEvents for Cigna Commercial HH and DME.

Diagram



Repo :- MemberDocumentServices

(https://dev.azure.com/eviCoreDev/ImageOneDomainApps/_git/MemberDocumentServices/branches Connect your Azure DevOps account)

Technology Stack

- **Framework**: .NET 9.0 (Latest LTS)
- **Architecture Pattern**: Clean Architecture + Microservices
- **Database**: SQL Server with Entity Framework Core 9.0.4
- **Message Broker**: Apache Kafka (Xplatform.Kafka 10.3.2)
- **Containerization**: Docker with multi-stage builds
- **Testing**: XUnit with Moq for mocking
- **Logging**: Application Insights + Serilog
- **Deployment**: Kubernetes-ready with Helm charts

External Systems:

- **Cigna Insurance Platform - Document upload and processing**
- **Kafka Event Streams - Real-time event processing**
- **SQL Server Database - Data persistence**
- **Email System - Notification services**
- **Application Insights - Monitoring and logging**

Hosting Model

- **Containerized deployment for AKS**

Context :-

Clinicians at Cigna are not able to view auth clinical documentation in their provider system (ProviderLink) in reviewing the authorization created in eviCore platform. Therefore, ep has developed capability to send clinical documentation for a Cigna commercial auth (HH, DME) to ProviderLink systematically. In order to migrate Cigna commercial program in ImageOne, this capability also developed in ImageOne platform.

Requirement

We have to send clinical documents based on below criteria's

- InsCarrier should be CIGNA
- InsPlanCode should be COMMERCIAL
- Program (CompanyID) should be HomeHealth/DME
- CIGNA Send 278R request and schema having clinicalReviewDeterminationCode : "A4".

Implementation

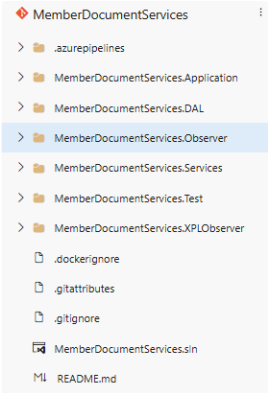
- We created a background service that runs continuously and consumes Kafka messages.
- This service is deployed on AKS.
- There are 2 different services which will be consumer 2 different kafka topics
  - XplObjectValetConsumer
  - AuthorizationRequest278Consumer
- **XplObjectValetConsumer** :- All the documents uploaded from ImageOne and CarrierWeb uploaded on "**XPLObjectValetEvents**" kafka topic. This consumer will access the Kafka topic "**XPLObjectValetEvents**" and read all the messages one by one. Internally it checks if the message containing episodeid matched the criteria of **CIGNA ⇒ COMMERCIAL ⇒ HH/DME** then it will create new entry in the "**ProviderLinkClinicalDocs**" table of pre-auththin database with **IsProcessed** flag 0.
- **AuthorizationRequest278Consumer** :- Whenever Cigna needs clinical document they will send 278R request to the Kafka Topic "**EDIIntake.V1**". This consumer will Access the Kafka topic "EDIIntake.V1" and reads the messages. Internally it checks the criteria CIGNA ⇒ COMMERCIAL ⇒ HH/DME ⇒ clinicalReviewDeterminationCode = "a4" and if condition matches then it fetch unprocessed (IsProcess = 0) records from ProviderLinkClinicalDocs table and sends documents to CIGNA Provider link portal and set IsProcess = 1.
- If there will be any failure during sending document to CIGNA ProviderLink then email will gets trigger with the Failed Case-id.

```
## 🏗️ **System Architecture Overview**

### **Core Processing Flow**
graph TD
    A[Kafka: 278R Authorization] --> B[Observer Service]
    B --> C[Authorization Processor]
    C --> D[Database: Get Clinical Docs]
    C --> E[Object Valet: Get File Content]
    C --> F[Cigna Upload Client]
    F --> G[Cigna API]
    C --> H[Email Service]

    I[Kafka: XPL Events] --> J[XPL Observer]
    J --> K[XPL Processor]
    K --> L[Database: Insert/Update Docs]
```

Project Structure



AuthorizationRequest278RProcessor

```
**Location**: `MemberDocumentServices.Services/ServiceImplementation/ProviderAuthorizationRequestKakfaProcessor.cs`

**Key Logic Flow**:

public async Task<CancellationToken> ProcessMessage(Authorization278Request kafkaMessage, ConsumeResult<Ignore, string> MessageDetails, CancellationToken Token)
{
    // 1. DECODE BASE64 PAYLOAD
    byte[] decodedBytes = Convert.FromBase64String(kafkaMessage.Payload);
    string decodedStr = Encoding.UTF8.GetString(decodedBytes);

    // 2. DESERIALIZE AUTHORIZATION MESSAGE
    AuthorizationRequest278R message = JsonConvert.DeserializeObject<AuthorizationRequest278R>(decodedStr);
    string episodeId = message?.authorization?.transactionId ?? "";

    // 3. VALIDATE REQUEST
    if (IsValidRequest(message))
    {
        // 4. GET CLINICAL DOCUMENTS FROM DATABASE
        var providerLinkDocs = await _clinicalDocs.GetClinicalDocumentsByEpisodeId(episodeId);

        // 5. PROCESS EACH DOCUMENT
        foreach (ProviderLinkClinicalDocs pd in providerLinkDocs.Where(d => d.ObjectId != "0"))
        {
            // 6. GET FILE CONTENT FROM OBJECT VALET
            FileDownloadResponse fileToReturn = await _filehandler.GetDocumentFileByObjectId(ObjectId);

            // 7. FILTER OUT SYSTEM FILES (_rendered.pdf, _hocr.txt, OCR results)
            if (!IsSystemGeneratedFile(uploadedFileName))
            {
                // 8. CREATE CIGNA METADATA
                CignaUploadMetadata metadata = SetCignaUploadMetadata(fileToReturn, message, uploadedFileName);

                // 9. UPLOAD TO CIGNA
                await _cignaUploadClient.UploadAsync(fileToReturn.Content, metadata, pd);
            }
        }
    }
}
```

```
**Critical Business Rules**:
- **File Filtering**: Excludes files ending with `_rendered.pdf`, `_hocr.txt`, and OCR text results
- **Error Handling**: File size exceptions trigger email notifications
- **Transaction Safety**: Each document upload is wrapped in try-catch blocks
- **Status Tracking**: Updates `IsProcessed` flag in database after successful upload

#### **B. IsValidRequest Method Logic**

private bool IsValidRequest(AuthorizationRequest278R message)
{
    // Validates required fields for Cigna processing
    return !string.IsNullOrEmpty(message?.authorization?.InsuranceCarrier) &&
           message.authorization.InsuranceCarrier.Equals("CIGNA") &&
           !string.IsNullOrEmpty(message?.authorization?.serviceType) &&
           HasRequiredMemberInfo(message.authorization);
}
```

XplObjectValetProcessor

```
**Location**: `MemberDocumentServices.Services/ServiceImplementation/XPLObjectValetkafkaProcessor.cs`

**Key Logic Flow**:
public async Task<CancellationToken> ProcessMessage(XplEvent xplObjectValetEvent, ConsumeResult<Ignore, string> MessageDetails, CancellationToken Token)
{
    // 1. VALIDATE XPL EVENT
    if (xplObjectValetEvent?.SystemOfRecordContextKey?.Length == 10)
    {
        // 2. CHECK IF CASE EXISTS IN AUTHORIZATION TABLE
        bool isCaseAvailable = await _clinicalDocs.GetCaseInfo(xplObjectValetEvent.SystemOfRecordContextKey);

        if (isCaseAvailable)
        {
            // 3. CHECK IF DOCUMENT ALREADY EXISTS
            var clinicalDocs_db = await _clinicalDocs.GetClinicalDocumentsByObjectId(xplObjectValetEvent.ObjectId);

            // 4. INSERT NEW DOCUMENT IF NOT EXISTS
            if (clinicalDocs_db.ObjectId == null)
            {
                ProviderLinkClinicalDocs clinicalDocs = BindProviderLinkDocs(xplObjectValetEvent);
                await _clinicalDocs.InsertClinicalDocuments(clinicalDocs);
```

```
            }
        }
    }
}
```

ClinicalDocs Updates

```
#### **ClinicalDocs Repository**
**Location**: `MemberDocumentServices.DAL/Implementation/ClinicalDocs.cs`

**Key Methods**:

// GET UNPROCESSED DOCUMENTS FOR EPISODE
public async Task<List<ProviderLinkClinicalDocs>> GetClinicalDocumentsByEpisodeId(string episodeId)
{
    return await _context.providerLinkClinicalDocs
        .Where(x => x.EpisodeId == episodeId && !x.IsProcessed)
        .ToListAsync();
}

// VALIDATE CIGNA COMMERCIAL CASES
public async Task<bool> GetCaseInfo(string episodeID)
{
    var res = await _context.tblAuthorization
        .Where(x => x.EpisodeID.Equals(episodeID) &&
                   x.InsCarrier.Equals("CIGNA") &&
                   x.InsPlanCode.Equals("COMMERCIAL") &&
                   (x.CompanyID.Equals(17) || x.CompanyID.Equals(15))) // HomeH or DME
        .ToListAsync();
    return res.Any();
}

// UPDATE PROCESSING STATUS
public async Task<bool> UpdateIsProcessed(ProviderLinkClinicalDocs clinicalDoc)
{
    _context.Update(clinicalDoc);
    await _context.SaveChangesAsync();
    return true;
}
```

CignaDocumentUploadClient

```
**Location**: `MemberDocumentServices.Services/ServiceImplementation/CignaDocumentUploadClient.cs`

**Upload Logic**:
public async Task UploadAsync(byte[] documentContent, CignaUploadMetadata metadata, ProviderLinkClinicalDocs providerLinkClinicalDocs)
{
    // 1. VALIDATE REQUIRED METADATA
    if (string.IsNullOrEmpty(metadata.MemberId) ||
        string.IsNullOrEmpty(metadata.MemberFirstName) ||
        string.IsNullOrEmpty(metadata.MemberLastName) ||
        metadata.MemberDateOfBirth == DateTime.MinValue)
        throw new CignaDocumentUploadException("Required information not available");

    // 2. CHECK FILE SIZE LIMIT
    if (documentContent.Length < _maxUploadFileSize)
    {
        // 3. UPLOAD TO CIGNA API
        await _api.UploadAsync(documentContent, _clientSettings.PostboxId, metadata, _logger);

        // 4. UPDATE DATABASE STATUS
        providerLinkClinicalDocs.IsProcessed = true;
        await _clinicalDocs.UpdateIsProcessed(providerLinkClinicalDocs);
    }
    else
    {
        // 5. HANDLE FILE SIZE EXCEEDED
        providerLinkClinicalDocs.IsProcessed = true;
        await _clinicalDocs.UpdateIsProcessed(providerLinkClinicalDocs);
        throw new CignaDocumentSizeExceededException("File cannot be uploaded to provider link. File size too large");
    }
}
```

DocFileHandler

```
**Location**: `MemberDocumentServices.Services/ServiceImplementation/DocFileHandler.cs`

**File Retrieval Logic**:

public async Task<FileDownloadResponse> GetDocumentFileByObjectId(long ObjectId)
{
```

```
    // 1. GET OAUTH TOKEN
    string AccessTokenURLAsKey = "OVAccessToken";
    var token = await apiTokenCacheClient.GetApiToken(AccessTokenURLAsKey);

    // 2. CREATE HTTP REQUEST
    var GetFileByObjectId = config.GetSection("OAuthSettings:GetFileByObjectId").Value;
    var request = new HttpRequestMessage()
    {
        RequestUri = new Uri(GetFileByObjectId + ObjectId.ToString()),
        Method = HttpMethod.Get
    };

    // 3. ADD HEADERS
    request.Headers.Add("ObjectValetApplication", ObjectValetApplication);
    request.Headers.Add("Authorization", "Bearer " + token);

    // 4. DOWNLOAD FILE
    HttpResponseMessage response = await client.SendAsync(request);
    if (response.IsSuccessStatusCode)
    {
        byte[] byteArray = await response.Content.ReadAsByteArrayAsync();
        string fileName = response.Headers.GetValues("Filename").FirstOrDefault() ?? string.Empty;
        string description = response.Headers.GetValues("Description").FirstOrDefault() ?? string.Empty;

        return new FileDownloadResponse
        {
            Content = byteArray,
            FileName = fileName,
            Description = description,
        };
    }
}
```

ApiTokenCacheClient

```
**Location**: `MemberDocumentServices.Services/ServiceImplementation/ApiTokenCacheClient.cs`

**Token Caching Logic**:

public async Task<string?> GetApiToken(string api_name)
{
    // 1. CHECK CACHE
    var accessToken = GetFromCache(api_name);

    // 2. VALIDATE EXPIRATION
    if (accessToken != null && accessToken.ExpiresIn > DateTime.UtcNow)
    {
        return accessToken.Access_token;
    }

    // 3. GET NEW TOKEN
    var newAccessToken = await GetOAuthToken(api_name);

    // 4. CACHE NEW TOKEN
    AddToCache(api_name, newAccessToken);

    return newAccessToken.Access_token;
}
```

EmailService

```
**Location**: `MemberDocumentServices.Services/ServiceImplementation/EmailService.cs`

**Email Logic**:

public async Task<bool> SendEmail(string emailSubject, string emailBody)
{
    if (!_emailIsEnabled) return false;

    using var mail = new MailMessage();
    foreach (var sender in _toSendList) mail.To.Add(sender);

    mail.From = new MailAddress(_from);
    mail.Subject = emailSubject;
    mail.Body = emailBody;
    mail.IsBodyHtml = true;
    mail.Priority = MailPriority.High;

    using var smtp = new SmtpClient(_smtpServer);
    smtp.UseDefaultCredentials = true;
    await smtp.SendMailAsync(mail);
```

```

    return true;
}
```

Configuration (You will get actual values in variable group)

```
#### **Kafka Configuration**
"AuthorizationRequest278R": {
    "TopicName": "__TopicName__",
    "ConsumerGroup": "__ConsumerGroup__",
    "SslCaLocation": "__SSlCaLocation__",
    "SecurityProtocol": "__SecurityProtocol__",
    "SaslMechanism": "__SSALMechanism__",
    "KafkaNodes": "__KafkaNodes__",
    "SaslUsername": "__SASLUserName__",
    "SaslPassword": "__SASLPassword__",
    "IsConsumerSecured": "__IsConsumerSecured__",
    "Enabled": "__KafkaConsumerEnabled__"
}


#### **Cigna Upload Configuration**

"CignaUpload": {
    "BaseUrl": "__CignaBaseURL__",
    "Username": "__CignaUserName__",
    "Password": "__CignaPassword__",
    "FacilityId": "__FacilityId__",
    "PostboxId": "__PostboxId__",
    "MaxUploadFileSizeInBytes": "__MaxUploadFileSizeInBytes__"
}

#### **OAuth Settings**
"OAuthSettings": {
    "AccessTokenUrl": "__AccessTokenURL__",
    "ScopeUrl": "__ScopeURL__",
    "ClientId": "__ClientId__",
    "ClientSecret": "__ClientSecret__",
    "GetFileByObjectId": "__GetFileByObjectId__",
    "ObjectValetApplication": "__ObjectValetApplication__"
}
```

Service Architecture

```
#### **Observer Service (Authorization 278R)**
- **Purpose**: Processes authorization requests from Kafka
- **Triggers**: Kafka messages on 278R topic
- **Output**: Cigna document uploads + database updates

#### **XPL Observer Service (Object Lifecycle)**
- **Purpose**: Tracks document lifecycle events
- **Triggers**: Kafka messages on XPL topic
- **Output**: Database document records
```

Bussiness Rules and Constraints

```
### **Document Processing Rules**
1. **Insurance Filter**: Only processes CIGNA Commercial plans
2. **Company Filter**: Only HomeH (17) or DME (15) companies
3. **File Exclusions**: Skip `_rendered.pdf`, `_hocr.txt`, OCR results
4. **Processing Status**: Mark as processed after upload attempt (success or failure)

### **Error Handling Strategy**
1. **File Size Exceeded**: Email notification + mark as processed
3. **Invalid Data**: Skip record with logging

### **Security Considerations**
1. **Certificate-based Kafka Authentication**: SSL/TLS with client certificates
2. **OAuth Token Management**: Automatic refresh with memory caching
3. **Basic Authentication**: Cigna API uses Base64 encoded credentials
4. **TLS Configuration**: Backward compatibility for legacy systems
```

Release Pipeline Details

AuthorizationRequest278R

Authorizationrequest278rconsumer_develop

Authorizationrequest278rconsumer_main

Authorizationrequest278rconsumer_release

XPLObjectValet

Xplobjectvaletcosumer_develop

Xplobjectvaletcosumer_main

Xplobjectvaletcosumer_release

Application Insight Details

- **AuthorizationRequest278R**

  - dv1-eu2-ioauthorizationrequest278rconsumer-appi
  - in1-eu2-ioauthorizationrequest278rconsumer-appi
  - in2-eu2-ioauthorizationrequest278rconsumer-appi
  - ld1-eu2-ioauthorizationrequest278rconsumer-appi
  - pd1-eu2-ioauthorizationrequest278rconsumer-appi
  - ut1-eu2-ioauthorizationrequest278rconsumer-appi

- **XPLObjectValet**

  - dv1-eu2-ioxplobjectvaletcosumer-appi
  - in1-eu2-ioxplobjectvaletcosumer-appi
  - in2-eu2-ioxplobjectvaletcosumer-appi
  - ld1-eu2-ioxplobjectvaletcosumer-appi
  - pd1-eu2-ioxplobjectvaletcosumer-appi
  - ut1-eu2-ioxplobjectvaletcosumer-appi

KeyVault Details

Name ↑		Type	Resource Group	Location	Subscription
dv1eu2imageone-aks-kv	···	Key vault	dv1-eu2-imageone-kv-rg	East US 2	eh-imageone-dev
in1eu2imageone-aks-kv	···	Key vault	in1-eu2-imageone-kv-rg	East US 2	eh-imageone-test
in2eu2imageone-aks-kv	···	Key vault	in2-eu2-imageone-kv-rg	East US 2	eh-imageone-test
ld1eu2imageone-aks-kv	···	Key vault	ld1-eu2-imageone-kv-rg	East US 2	eh-imageone-test
pd1eu2imageone-aks-kv	···	Key vault	pd1-eu2-imageone-kv-rg	East US 2	eh-imageone-prod
ut1eu2imageone-aks-kv	···	Key vault	ut1-eu2-imageone-kv-rg	East US 2	eh-imageone-prod

ArgoCD Url

https://eh-aks-1-dev.innovate.lan/settings/repos?addRepo=false

https://eh-aks-1-intg.innovate.lan/settings/repos?addRepo=false

https://eh-aks-1-prod.innovate.lan/settings/repos?addRepo=false